



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Uniform-Cost Search orders the frontier purely by $g(n)$, the actual path cost accumulated from the start node. It always expands the cheapest-so-far node, with no notion of where the goal actually is, so it explores uniformly outward in all directions until it stumbles onto the goal.

A* Search orders the frontier by $f(n) = g(n) + h(n)$, combining the real cost so far with an estimate of the remaining cost to the goal supplied by the heuristic. This lets A* favour nodes that are both cheap to reach and appear close to the goal, so it expands far fewer irrelevant nodes and drives its search directly toward the goal instead of spreading out uniformly like UCS.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

In this grid, the agent can only move Up, Down, Left, or Right, and every move costs exactly 1. Manhattan distance, $|x_1 - x_2| + |y_1 - y_2|$, is exactly the number of horizontal and vertical steps needed to reach the goal if no walls existed. Since walls can only force detours that add extra steps, Manhattan distance can never overestimate the true remaining cost; it is a lower bound on the actual path cost. A heuristic that never overestimates the true cost to the goal is, by definition, admissible.

If the heuristic were not admissible (i.e., it sometimes overestimated the remaining cost), A* could be misled into treating a node that lies on the true optimal path as if it were more expensive than it really is. The algorithm might then expand and return a path through a node with a lower true cost, meaning A* would no longer be guaranteed to find the optimal (shortest) path; it could return a suboptimal solution, even though it would typically still terminate and find some solution.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan distance would stop being admissible under 8-way movement. With diagonal moves allowed, the agent can cover both an x-step and a y-step in a single move of cost 1, whereas Manhattan distance still counts them as two separate unit steps. This means Manhattan distance would overestimate the true cheapest cost whenever a diagonal shortcut exists, so it would no longer be a guaranteed lower bound.

The appropriate metric depends on how diagonal moves are costed:

- * If a diagonal move costs the same as an orthogonal move (cost 1), the Chebyshev distance, $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$, is the admissible choice, since it correctly counts the minimum number of unit moves (straight or diagonal) needed.
- * If a diagonal move costs $\sqrt{2}$ (true geometric distance), Euclidean distance, $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$, becomes the admissible and consistent choice, since it never overestimates the true straight-line travel cost.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Distance to only the single nearest food item ignores how far the agent will still have to travel after eating it, so it can lead A* toward a food pellet that is close now but sits in an isolated corner, forcing a long detour to reach the rest. A stronger heuristic needs to account for the cost of visiting all remaining food, not just one item. Two practical options:

Option A: Nearest-food-plus-spread (cheap, informed upper bound on structure)

$h(n)$ = distance to the nearest remaining food + the maximum pairwise Manhattan distance between all remaining food items. This estimates "travel to the food cluster" plus "cross the spread of the cluster once you're there," giving a better sense of the true remaining effort than a single nearest-food distance, while still being cheap to compute.

Option B: Minimum Spanning Tree (MST) heuristic (stronger, standard for multi-goal search)

Build a graph over the remaining food positions (edge weights = Manhattan distance between pairs), compute its Minimum Spanning Tree, and set $h(n)$ = distance from the agent's current position to the nearest food + the total weight of the MST connecting all remaining food. The MST weight is a lower bound on the cost of visiting every remaining food item, since any tour that eats all the food must connect them at least as expensively as their MST. This heuristic remains admissible (it never overestimates, because walls only add cost) and scales well because the MST only needs to be rebuilt when a food item is eaten, not on every node expansion.

Either option gives A* a far more accurate estimate of the true remaining cost than "distance to closest food alone," which should noticeably reduce the number of nodes explored while still finding an efficient path through all the food.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING

